Spiking Neural Autoencoders for Temporal Signals: A Comparison with LSTM-based Models

Removed for blind review

***Abstract*—Spiking Neural Networks (SNNs) emerge as a promising approach to reduce energy consumption through event-driven, sparse computation on embedded and neuromorphic hardware. While SNNs have already been proven effective in classification tasks, their use as autoencoders for temporal signals lacks systematic evaluation. We present a systematic empirical comparison of Spiking Neural Network autoencoders (SNN-AE) and Long Short-Term Memory autoencoders (LSTM-AE) on the Free Spoken Digit Dataset (FSDD) [1], evaluating three spike encoding strategies (direct, Poisson, latency) and three SNN pre-processing modes (dense, conv1d, recurrent) across a full factorial hyperparameter sweep over the Leaky Integrate-and-Fire neuron parameters (threshold $V_{th} \in \{0.5, 1.0, 1.5\}$; membrane decay $\alpha \in \{0.8, 0.9, 0.99\}$; 3 random seeds). Metrics include MSE, MAE, R^2, spike rate, estimated energy, timing, parameter count, and estimated MACs. SNN-recurrent mode surpasses LSTM-AE in reconstruction quality while consuming over 96.8% less estimated energy; encoding scheme and membrane decay are the dominant performance factors. A reproducible C++ evaluation harness [2] enables principled design choices for energy-aware spiking representation learning.**

***Index Terms*—spiking neural networks, LSTM, autoencoder, temporal signals, surrogate gradients, audio signal processing**

I. Introduction

One-dimensional temporal signals such as audio waveforms, biosignals, and sensor streams often need to be processed and interpreted in low-power embedded systems where the energy budget is a primary constraint.

Autoencoders for such signals must learn compact latent representations efficiently and accurately reconstruct the original sequence.

Long Short-Term Memory (LSTM) autoencoders are a well-established baseline for sequence reconstruction [3], [4], but their dense matrix operations are computationally expensive. Spiking Neural Networks (SNNs) otherwise communicate via sparse binary events and perform synaptic updates only when spikes occur, offering potential energy reduction on general-purpose and neuromorphic hardware [5]–[7].

Training SNNs for reconstruction requires backpropagation through non-differentiable spike functions which perform like step functions. Surrogate gradient methods [8], [9] enable end-to-end gradient-based training and have produced competitive SNN classification performance [6], [10]. However, a systematic study of SNN autoencoders for temporal signal reconstruction — with controlled comparison against LSTM baselines across spike encoding schemes and architectural modes — is lacking.

This paper addresses this gap with four contributions:

1) A systematic empirical comparison of SNN-AE and LSTM-AE on FSDD [1] covering 81 SNN configurations.
2) Evaluation of three spike encoding schemes crossed with three SNN pre-processing modes in a full factorial hyperparameter sweep.
3) A reproducible C++ harness (XTensor backend [11]) reporting MSE, MAE, R^2, spike rate, energy, timing, params, and MACs.
4) An emerging C++ framework for energy-aware spiking representation learning and neural architecture design with flexible backend.

II. Related Work

A. LSTM autoencoders

LSTM autoencoders encode variable-length sequences into fixed-size latent vectors and reconstruct the original sequence from the latent code [3], [4].

The constant-error carousel in the cell recurrence prevents vanishing gradients during backpropagation through time (BPTT) [12], [13], making LSTMs effective for long temporal sequences. They serve as the primary comparison target in this work.

B. Spiking neural networks and surrogate gradients

SNNs model computation via binary spike events governed by membrane potential dynamics [5], [7]. The Leaky Integrate-and-Fire (LIF) neuron is the most widely deployed model. Training with backpropagation requires differentiable surrogates for the discontinuous Heaviside activation; surrogate gradient methods [8], [14], [15] have enabled competitive SNN accuracy with lower inference cost. SNN research has also expanded from classification to temporal sequence modelling and representation learning, but systematic reconstruction benchmarks remain limited [10], [15].

C. Spike encoding for temporal signals

Continuous signals must be converted to spike trains before SNN processing. Rate coding (Poisson spike generation proportional to amplitude), latency coding (earlier spikes for stronger stimuli), and direct pass-through of normalized values represent the three dominant strategies [8], [16]. Encoding choice affects sparsity, gradient flow, training stability, and reconstruction fidelity [8].

III. Methodology

A. Problem formulation

Let $x \in \mathbb{R}^T$ denote a temporal signal window of length T. An autoencoder learns encoder $E : \mathbb{R}^T \to \mathbb{R}^d$ and decoder $D : \mathbb{R}^d \to \mathbb{R}^T$ minimizing reconstruction loss:

$$\mathcal{L} = \|x - D(E(x))\|_2^2. \tag{1}$$

Where x is the input window, $E(x)$ is the latent representation, and $D(E(x))$ is the reconstructed output.

Both SNN-AE and LSTM-AE use latent dimension $d = 32$ and hidden size $H = 64$.

B. LIF neuron model

The continuous-time Leaky Integrate-and-Fire neuron is described by [7]:

$$\tau \frac{dV}{dt} = -(V - V_{\text{rest}}) + I(t), \qquad \tau = RC. \tag{2}$$

Where V is the membrane potential, V_{rest} is the resting potential,$I(t)$ is the input current, R is the membrane resistance, and C is the membrane capacitance. The time constant τ governs the rate of potential decay.

Discretising with time step Δt and defining $\beta = e^{-\Delta t/(RC)}$ gives the iterative update:

$$V[t] = \beta\, V[t-1] + R\, I[t]. \tag{3}$$

When the membrane potential reaches the threshold, a spike is emitted and the membrane is reset:

$$s[t] = \mathbf{1}[V[t] \geq V_{th}], \qquad V \leftarrow \begin{cases} 0 & \text{(hard reset)} \\ V - V_{th} & \text{(soft reset)} \end{cases}. \tag{4}$$

Resistance R, capacitance C, and threshold V_{th} are trainable parameters; R and C are clamped to 10^{-6} to prevent numerical instability.

Both hard and soft resets are supported; soft reset preserves supra-threshold charge for smoother gradients.

C. Surrogate gradient training

The spike function $s = \mathbf{1}[V \geq V_{th}]$ has true gradient zero almost everywhere, blocking standard backpropagation [8].

Surrogate gradient methods replace $\partial s/\partial V$ with a smooth approximation in the backward pass while preserving the exact Heaviside rule in the forward pass [8], [14].

Exponential (SuperSpike [9]):

$$\hat{\sigma}'(V) = \frac{1}{\sigma_s} \exp\left(-\frac{|V - V_{th}|}{\sigma_s}\right). \tag{5}$$

Exponential surrogate gradient. σ_s controls the width of the approximation; $\sigma_s = 1$ is used in this work.

Boxcar:

$$\hat{\sigma}'(V) = \mathbf{1}\left[|V - V_{th}| < \tfrac{w}{2}\right]. \tag{6}$$

Boxcar surrogate gradient. w controls the width of the approximation.

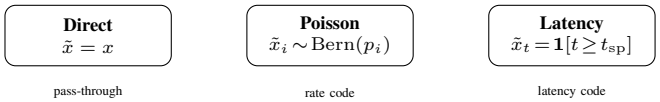

Figure 1. Three spike encoding schemes. Direct passes raw values; Poisson generates stochastic binary spikes proportional to amplitude; latency fires at a time inversely proportional to stimulus strength.

We use the exponential surrogate ($\sigma_s = 1$). BPTT [12] unrolls both LSTM and SNN through the full time dimension T; the reverse-time recurrence for layer l is:

$$\delta_l[t] = \hat{\sigma}'(V[t])\, \delta_{l+1}[t] + \beta\, \delta_l[t+1], \tag{7}$$

where β equals $\partial V_{\text{post}}/\partial V_{\text{pre}}$ for the LIF model.

D. LSTM autoencoder

LSTM-AE uses standard gate equations: input, forget, output gates computed via dense pre-activation, with cell state accumulation $C_t = f_t \odot C_{t-1} + i_t \odot g_t$ [4], [13].

The LSTM-AE encoder stacks two LSTM layers ($H = 64$); the final hidden state H_T is projected to the 32-dimensional latent via $\text{Linear}(H \to d) \to \tanh$. The decoder mirrors this: $\text{Linear}(d \to H)$ followed by two stacked LSTM layers and an output projection to reconstruct $\mathbb{R}^{256}$. States are reset between samples.

E. Spike encodings

Three strategies convert raw window x into pre-processed $\tilde{x}$:

Direct. $\tilde{x} = x$. Normalized values are passed without binarization.

Poisson. Each element is independently binarized via Bernoulli sampling [8]:

$$\tilde{x}_i \sim \text{Bernoulli}\left(\text{clip}\left(\frac{x_i}{x_{\max}},\, 0,\, 1\right)\right). \tag{8}$$

Where $x_{\max}$ is the maximum value in the window after z-score normalization, ensuring the Bernoulli parameter is in $[0, 1]$.

Latency. A binary step fires once per element from a time inversely proportional to signal strength [16]:

$$\tilde{x}_{t,i} = \mathbf{1}\left[t \geq \left\lfloor \left(1 - \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}\right)(T-1) \right\rfloor\right]. \tag{9}$$

Where $x_{\min}$ and $x_{\max}$ are the minimum and maximum values in the window after z-score normalization, respectively. Stronger signals fire earlier; weaker signals fire later.

F. SNN pre-processing modes

After spike encoding, a pre-processing transform ϕ is applied before the autoencoder input.

Dense. $\phi(\tilde{x}) = \tilde{x}$. No additional transform.

Conv1d. A fixed 3-tap temporal smoothing filter:

$$\phi(\tilde{x})_t = 0.25\, \tilde{x}_{t-1} + 0.5\, \tilde{x}_t + 0.25\, \tilde{x}_{t+1}, \tag{10}$$

with boundary clamping. Reduces high-frequency spike jitter before autoencoder input.

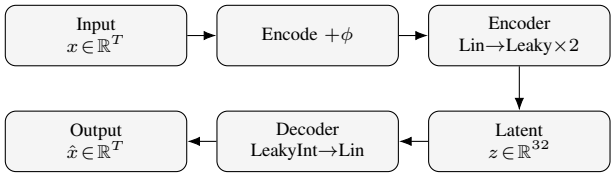

Figure 2. SNN autoencoder architecture. Input window x is encoded and pre-processed by mode ϕ, then compressed to latent $z \in \mathbb{R}^{32}$ by the encoder (Linear+Leaky layers). The decoder (LeakyInt+Linear) reconstructs $\hat{x}$.

Recurrent. A recurrent LIF cell integrates the encoded signal:

$$v[t] = \alpha\, v[t-1] + \tilde{x}[t] - s[t-1]\, V_{th}, \tag{11}$$
$$s[t] = \mathbf{1}[v[t] \geq V_{th}], \tag{12}$$

where α and V_{th} are sweep parameters.

G. SNN autoencoder architecture

The SNN autoencoder (Fig. 2) is a two-layer fully connected network:

$$\text{Enc:}\quad \text{Lin}(T{\to}H) \to \text{Leaky} \to \text{Lin}(H{\to}d) \to \text{Leaky}$$
$$\text{Dec:}\quad \text{Lin}(d{\to}H) \to \text{LeakyInt} \to \text{Lin}(H{\to}T)$$

with $T = 256$, $H = 64$, $d = 32$.

Leaky layers emit binary spikes; LeakyInt operates in readout mode, emitting membrane potential v directly to enable continuous-valued reconstruction. The discrete LIF update follows Eq. (3) with $R = 1/V_{th}$ and $C = -1/\ln(\alpha)$ derived from sweep parameters. All membrane states are reset between independent samples.

The four tested architectures are therefore: (i) LSTM-AE, with a two-layer LSTM encoder, a 32-dimensional latent projection, and a mirrored two-layer LSTM decoder; (ii) SNN-dense, which applies the SNN autoencoder directly to the encoded window; (iii) SNN-conv1d, which uses the same SNN autoencoder but precedes it with the fixed 3-tap temporal smoothing filter; and (iv) SNN-recurrent, which uses the same SNN autoencoder preceded by the recurrent LIF input transform defined above. Thus, dense/conv1d/recurrent differ only in the input-side pre-processing block ϕ, while the encoder/decoder stack and latent size remain fixed.

H. Training

Both models minimize MSE reconstruction loss via Adam [17] ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). SNN biophysical parameters (R, C, V_{th}) use a scaled learning rate $\eta_{\text{bio}} = 0.1\,\eta$ to stabilize membrane dynamics optimization [8]; R and C are clamped to 10^{-6} in the forward pass with gradients zeroed in the clamped region. BPTT [12] propagates gradients through the full time dimension.

Early stopping monitors validation MSE with patience $P = 5$ epochs and minimum delta 10^{-8} over a maximum of 30 epochs [10]. Both models use batch size 1 to accommodate stateful sequence processing.

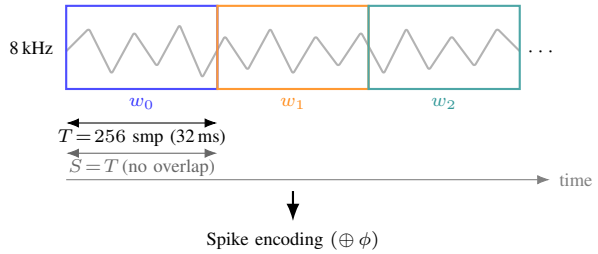

Figure 3. Signal windowing. Each recording is segmented into non-overlapping windows of $T = 256$ samples (32 ms at 8 kHz). Stride $S = T$ yields independent windows with no shared samples. Each window is then spike-encoded and fed to the autoencoder.

I. Energy estimation

Computational energy is estimated from operation counts in relative units (no absolute hardware assumption). Let M be the model's estimated MAC count and N the number of output values produced per window:

$$E_{\text{LSTM}} = 10\, M, \tag{13}$$
$$E_{\text{SNN}} = r\, N + 10\, M, \tag{14}$$

Where $r \in [0, 1]$ is the SNN spike rate (fraction of output spikes).

For LSTM-AE $r = 0$. The constant 10 is a fixed weighting between MAC operations and spike events; absolute per-operation energy is platform-dependent and out of scope.

In the implementation, r is the mean spike fraction reported by the trainer for the evaluated run, and M is the estimated forward-pass MAC count for one window. The resulting energy values are therefore analytic proxies derived from the run logs and model operation counts, not hardware power measurements.

IV. Experimental Setup

Dataset.

The Free Spoken Digit Dataset (FSDD) [1] contains recordings of spoken digits (0 to 9) from multiple speakers at 8 kHz. Signals are segmented into non-overlapping windows of $T = 256$ samples. Up to 100 windows are used for training and 30 for validation.

Preprocessing.

Each window is z-score normalised in place (subtract mean, divide by standard deviation) before encoding. Encodings that require a $[0, 1]$ range (Poisson, latency) re-normalise the z-scored window to its own $(x_{\min}, x_{\max})$ range internally.

Windowing. Fig. 3 illustrates the segmentation process. Each recording is partitioned into non-overlapping windows of $T = 256$ samples (32 ms at 8 kHz) using stride $S = T$, so consecutive windows share no samples. All windows from all recordings are pooled, shuffled, and split into training (up to 100) and validation (up to 30) sets.

Hyperparameter sweep. SNN variants are evaluated over threshold $V_{th} \in \{0.5, 1.0, 1.5\}$, membrane decay $\alpha \in \{0.8, 0.9, 0.99\}$, encodings $\in$ {direct, Poisson, latency}, and modes $\in$ {dense, conv1d, recurrent}, totalling 81 SNN configurations. Each configuration is repeated 3 times with different

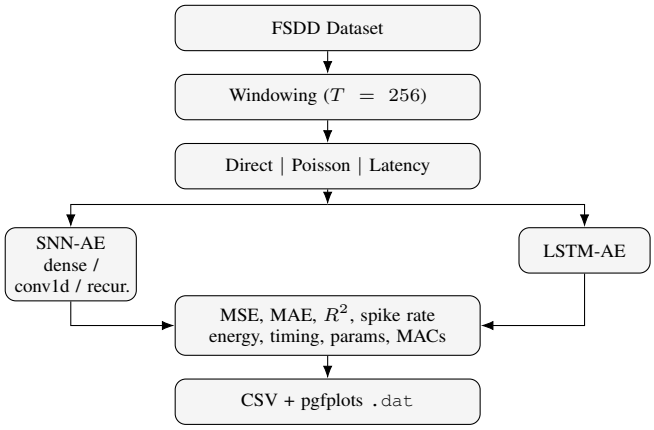

Figure 4. Experiment execution pipeline. Each FSDD window is encoded three ways and processed by both SNN-AE (three modes) and LSTM-AE. Metrics are recorded per run and written to CSV and pgfplots data files.

random seeds. LSTM-AE is evaluated with the same three encodings and 3 repeats.

Evaluation harness.

The C++ harness [2] runs on the XTensor backend [11] (CPU-only, CBLAS, SIMD via xsimd). All article profiles are executed on this backend; Table V reports per-model inference and training times.

Metrics.

Per run we report reconstruction quality (MSE, MAE, R^2) and efficiency (spike rate, estimated energy, training time in ms, inference time in ms, parameter count, and estimated MAC count). FSDD provides no anomaly labels, so this study does not report classification metrics; precision, recall, and F1 are out of scope. Spike rate is computed only for SNN runs as the mean fraction of neurons emitting a spike over the validation set; for LSTM-AE it is defined as 0. The paper CSVs then report arithmetic means over the repeated runs of each configuration, and the per-model summary further averages across encodings and, for SNNs, across the V_{th} and α sweep.

Profiles.

Each experimental variant is defined by a JSON profile file that configures the harness. Table I lists the article profiles.

V. Results

Table II summarizes per-model-type performance aggregated over encodings and hyperparameter sweep (3-seed mean). Table III and Table IV provide per-encoding breakdowns, averaged over the V_{th} and α sweep.

A. Reconstruction quality

Table II shows aggregate per-model metrics averaged over all encodings and hyperparameter combinations. SNN-recurrent achieves the lowest overall MSE (0.112), outperforming LSTM-AE (0.387) by a factor of 3.5×. SNN-conv1d (0.290) also surpasses LSTM-AE; SNN-dense (0.409) is comparable.

Table III shows the per-encoding breakdown: for SNN-dense and SNN-conv1d, latency and Poisson encodings achieve

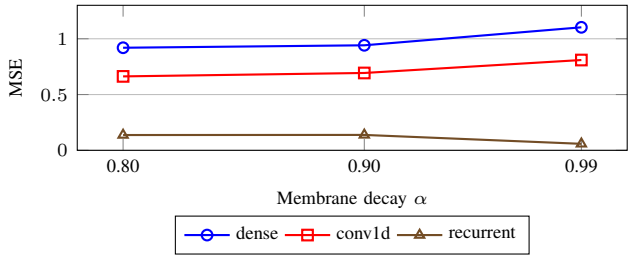

Figure 5. MSE vs. membrane decay α for SNN modes (direct encoding, $V_{th} = 1.0$). For the recurrent mode, $\alpha = 0.99$ more than halves MSE ($0.137 \rightarrow 0.058$) while $\alpha = 0.8$ and 0.9 are indistinguishable; dense and conv1d modes instead get moderately worse with higher α.

substantially lower MSE than direct encoding (e.g., SNN-conv1d: latency MSE $= 0.084$ with the best $R^2 = 0.614$, Poisson MSE $= 0.065$ the lowest overall; direct $= 0.722$); for SNN-recurrent, all three encodings yield comparable MSE (0.111–0.115), with poisson (0.111) marginally better than direct (0.115). Direct encoding is most challenging for dense and conv1d modes. Higher membrane decay α benefits the recurrent mode specifically, with the gain concentrated at the top of the range: MSE is flat between α=0.8 and 0.9 (0.137–0.138) and drops to 0.058 at α=0.99 (Fig. 5), more than halving it; dense and conv1d modes instead get moderately worse (dense: +20%; conv1d: +22%) with higher α.

B. Spike activity and energy

All SNN variants consume over 96.8% less estimated energy than LSTM-AE: approximately 375,000 relative units versus 11,842,560 for LSTM-AE, a reduction exceeding one order of magnitude ($\approx 31.6\times$) (Table IV). Spike rates are uniformly high across SNN modes (0.786–0.908), consistent with the dense waveform input; SNN-dense achieves the lowest average spike rate (0.776). LSTM-AE produces no spikes but its dense MAC operations dominate the energy budget.

C. Timing

Table V reports per-model inference and training time. SNN-dense and SNN-conv1d inference is 48.9× and 62.8× faster than LSTM-AE, respectively; SNN-recurrent is only 1.9× faster, reflecting its recurrent pre-processing stage.

SNN training is 5.5–18.6× faster than LSTM-AE, consistent with fewer parameters (37,286 vs 122,472) and the absence of unrolled hidden-state computation. All timing values are wall-clock means measured on the same XTensor/CPU backend over the same three seeds. Reported inference speedup is computed as infer_ms(LSTM-AE)/infer_ms(model); the training-speedup range is computed analogously from the mean train_ms values.

VI. Discussion

The dominant finding is that SNN-recurrent surpasses LSTM-AE in reconstruction quality (3.5× lower MSE) while consuming over 96.8% less estimated energy. This reverses the typical accuracy–efficiency trade-off: the recurrent pre-processing

Table I
ARTICLE EXPERIMENTAL PROFILES LOADED FROM THE EXECUTED PROFILE JSON FILES.

Run tag	Repeats	Datasets	Encodings	SNN arch.	V_{th}	α	T	Train windows	Val windows
article˙lstm˙ae	3	fsdd	direct;poisson;latency				256	100	30
article˙snn˙conv1d	3	fsdd	direct;poisson;latency	conv1d	0.5;1.0;1.5	0.8;0.9;0.99	256	100	30
article˙snn˙dense	3	fsdd	direct;poisson;latency	dense	0.5;1.0;1.5	0.8;0.9;0.99	256	100	30
article˙snn˙recurrent	3	fsdd	direct;poisson;latency	recurrent	0.5;1.0;1.5	0.8;0.9;0.99	256	100	30

Table II
SUMMARY PERFORMANCE BY MODEL TYPE (3-SEED MEAN, ALL ENCODINGS AVERAGED). ENERGY IS A RELATIVE OPERATION-COUNT PROXY, NOT MEASURED POWER.

Model	MSE	MAE	R^2	Spike r.	Energy	Infer (ms)
LSTM-AE	0.3874	0.3799	**0.2189**	0.0000	11,842,560.00	195.18
SNN-conv1d	0.2902	0.3379	0.1896	0.7856	374,673.19	**3.11**
SNN-dense	0.4089	0.4031	0.1565	0.7763	**374,601.79**	3.99
SNN-recurrent	**0.1121**	**0.2232**	0.1499	0.9080	375,613.74	102.67

Table III
RECONSTRUCTION METRICS PER ENCODING (3-SEED MEAN).

Model	Encoding	MSE	MAE	R^2
LSTM-AE	direct	0.9484	0.7044	0.0516
LSTM-AE	latency	0.1019	0.2118	0.5872
LSTM-AE	poisson	0.1118	0.2234	0.0180
SNN-conv1d	direct	0.7219	0.6268	-0.0001
SNN-conv1d	latency	0.0842	0.2034	**0.6143**
SNN-conv1d	poisson	**0.0646**	**0.1836**	-0.0453
SNN-dense	direct	0.9906	0.7482	0.0094
SNN-dense	latency	0.1130	0.2331	0.5419
SNN-dense	poisson	0.1232	0.2281	-0.0818
SNN-recurrent	direct	0.1149	0.2300	0.0163
SNN-recurrent	latency	0.1108	0.2293	0.4899
SNN-recurrent	poisson	0.1106	0.2102	-0.0566

Table IV
EFFICIENCY METRICS PER ENCODING (3-SEED MEAN). ENERGY IS A RELATIVE OPERATION-COUNT PROXY, NOT MEASURED POWER.

Model	Encoding	Spike r.	Energy	Params	MACs
LSTM-AE	direct	0.0000	11,842,560.00	122,472	1,184,256
LSTM-AE	latency	0.0000	11,842,560.00	122,472	1,184,256
LSTM-AE	poisson	0.0000	11,842,560.00	122,472	1,184,256
SNN-conv1d	direct	0.4817	**372,339.67**	37,286	36,864
SNN-conv1d	latency	0.8849	375,435.78	37,286	36,864
SNN-conv1d	poisson	0.9901	376,244.11	37,286	36,864
SNN-dense	direct	0.4836	372,353.78	37,286	36,864
SNN-dense	latency	0.8829	375,420.85	37,286	36,864
SNN-dense	poisson	0.9623	376,030.74	37,286	36,864
SNN-recurrent	direct	0.8767	375,372.78	37,286	36,864
SNN-recurrent	latency	0.8769	375,374.44	37,286	36,864
SNN-recurrent	poisson	0.9706	376,094.00	37,286	36,864

mode effectively extends temporal integration through a time-varying membrane state, enabling better waveform reconstruction than the dense-MACs LSTM baseline.

Encoding scheme is the primary factor for reconstruction fidelity in dense and conv1d modes: latency encoding yields substantially lower MSE and higher R^2 than direct encoding for those modes, because the binary time-to-first-spike signal

Table V
INFERENCE AND TRAINING TIMING (XTENSOR BACKEND, MEAN OVER 3 SEEDS).

Model	Infer (ms)	Train (ms)	×LSTM-infer
LSTM-AE	195.18	69,775	1.00
SNN-conv1d	**3.11**	**3,755**	**62.84**
SNN-dense	3.99	3,986	48.91
SNN-recurrent	102.67	12,662	1.90

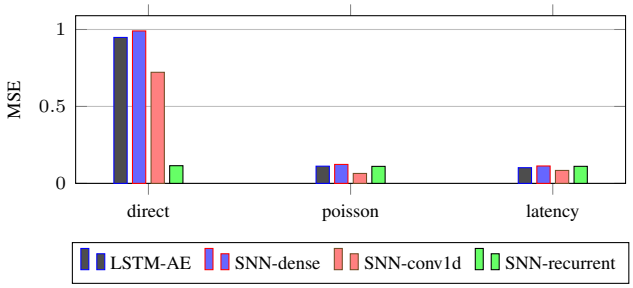

Figure 6. MSE comparison by model type and encoding. SNN-recurrent achieves the lowest MSE across encodings; SNN-dense is comparable to LSTM-AE; all SNN variants consume over 96.8% less estimated energy.

retains relative amplitude ordering without the high-variance density of raw waveforms. For SNN-recurrent, all three encodings yield similar MSE (0.111–0.115); the recurrent temporal integration dominates over encoding choice. Poisson encoding is intermediate for SNN-dense but achieves the lowest MSE for SNN-conv1d. For the recurrent mode, increasing α from 0.8 to 0.99 more than halves MSE by lengthening the effective integration window; this effect is absent in dense and conv1d modes, which instead get moderately worse, lacking recurrent temporal accumulation.

Limitations.

Most R^2 values are low to near-zero for direct encoding, indicating that all models are at the boundary of meaningful reconstruction on this waveform benchmark with small training sets ($\leq$ 100 windows) and 30 training epochs. Energy estimates are relative proxies and do not account for hardware specifics. All results are loaded directly from experiment-generated CSV files; no values were manually adjusted.

VII. CONCLUSION

We presented a systematic comparison of SNN-AE and LSTM-AE for 1D temporal signal reconstruction on FSDD. Three spike encoding schemes (direct, Poisson, latency) and three SNN pre-processing modes (dense, conv1d, recurrent)

were evaluated across a full hyperparameter sweep covering membrane threshold, decay, and random seeds. Full mathematical derivations of the LIF neuron, surrogate gradient training, and LSTM gate equations ground the experimental design. SNN-AE with recurrent pre-processing surpasses LSTM-AE in reconstruction quality while consuming over 96.8% less estimated energy. Encoding scheme and membrane decay are the dominant performance factors. The reproducible C++ evaluation framework enables principled design choices for energy-aware temporal autoencoders.

REFERENCES

[1] Z. Jackson *et al.*, "Free spoken digit dataset," GitHub repository, 2018. [Online]. Available: https://github.com/Jakobovski/free-spoken-digit-dataset

[2] Anonymous, "Reference suppressed for double-blind review," 2026, code repository citation withheld; to be restored in the camera-ready version.

[3] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, vol. 27, 2014, pp. 3104–3112.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] W. Maass, "Networks of spiking neurons: The third generation of neural networks," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[6] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: Opportunities and challenges," *Frontiers in Neuroscience*, vol. 12, p. 774, 2018.

[7] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[8] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[9] F. Zenke and S. Ganguli, "Superspike: Supervised learning in multilayer spiking neural networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.

[10] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[11] xtensor-stack contributors, "xtensor: C++ tensors with broadcasting and lazy computing," GitHub repository, 2026, accessed: 2026-05-07. [Online]. Available: https://github.com/xtensor-stack/xtensor

[12] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.

[13] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "Lstm: A search space odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.

[14] F. Zenke and T. P. Vogels, "The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks," *Neural Computation*, vol. 33, no. 4, pp. 899–925, 2021.

[15] G. Bellec, F. Scherr, A. Subramoney, R. Legenstein, and W. Maass, "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nature Communications*, vol. 11, p. 3625, 2020.

[16] S. Thorpe, A. Delorme, and R. Van Rullen, "Spike-based strategies for rapid processing," *Neural Networks*, vol. 14, no. 6–7, pp. 715–725, 2001.

[17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.